

Capítulo 2: La capa de aplicación

▼ Cliente-Servidor vs P2P

En **C-S** existe un host siempre activo, denominado servidor, que da servicio a las solicitudes de muchos otros hosts, llamados clientes.

Mientras que en **P2P** existe una mínima dependencia de infraestructura de servidores dedicados situados en centros de datos. Esta arquitectura explota la comunicación directa entre hosts, denominados **peers**.

▼ Servicios de transporte disponibles para las apps

Los servicios que puede ofrecer un protocolo de transporte son algunos de estos:

▼ Transferencia de datos fiables

Como ya se sabe, existe la pérdida de paquetes. En muchas apps esto no es tolerable, por lo tanto, es necesario garantizar que los datos enviados sean entregados de forma correcta y completamente.

▼ Tasa de transferencia (Throughput)

Es la velocidad **real** a la que el emisor le está entregando los datos al receptor. Como esto no necesariamente es constante, se divide a las apps en dos tipos:

- **Sensibles al ancho de banda:** que necesitan una velocidad mínima para funcionar, sino fallan.
- **Elásticas:** Se adaptan a la velocidad. Pueden tardar más o menos, dependiendo de eso.

▼ Temporizador (Timing)

Hay aplicaciones que tienen como prioridad el tiempo que tardan, más que cuanto mandan. Estas apps se las denominan **interactivas en tiempo real**.

▼ Seguridad

Un protocolo de transporte podría implementar servicios de seguridad tales como: **cifrado de datos, integridad de los datos y mecanismos de autenticación en el punto terminal.**

▼ TCP

- Es un protocolo **orientado a la conexión** y de **transferencia de datos fiables**.
- Dispone de un handshake para establecer la conexión y la conexión es full-duplex.
- Incluye un mecanismo de control de congestión
- Existe una versión de seguridad llamada **SSL (Secure Sockets Layer)**.

▼ UDP

- Es un protocolo **no orientado a la conexión** y de **transporte ligero**.
- Proporciona un servicio de transferencia de datos no fiable.
- Los paquetes no llegan en orden.

▼ HTTP

Es un protocolo de la capa de aplicación de la Web. Usa la arquitectura **C-S** y se comunican entre sí intercambiando mensajes **HTTP**.

HTTP utiliza **TCP** como capa de transporte. Por lo tanto, hay handshake por cada conexión.

Existen dos tipos de conexiones:

- **No persistentes:** para cada objeto que pide el cliente, se abre una nueva conexión **TCP** para luego cerrarla una vez respondido.
- **Persistentes:** se deja la conexión abierta hasta que ocurra un timeout.

HTTP usa el puerto **80**, mientras que su otra versión **HTTPS** usa el 443.

También usa algunos códigos de error para sus respuestas, tales como:

- 200 OK
- 400 Bad Request

- 404 Not Found

Entre otros.

Cookies

Se usan generalmente para decirle al servidor que dos peticiones tienen su origen en el mismo navegador web. Es una forma de identificación local que guarda el navegador para luego transmitírselo al servidor.

Web Caching

También llamada **servidor proxy**, es una entidad de red que satisface solicitudes HTTP en nombre de un servidor web de origen. Es un intermediario que guarda en cache, copias de los contenidos de Internet para no tener que ir a buscarlos todo el tiempo.

▼ CDN

Es una red de servidores distribuidos geográficamente que trabajan juntos para proporcionar entrega rápida de contenido de internet.

Es una solución técnica al problema de la **latencia** y el **cuello de botella** en el núcleo de la red.

La CDN pone copias del contenido en servidores estratégicos situados en el **extremo de la red** (edge).

▼ DNS

Definición

Los nombres de los hosts son mnemónicos, quiere decir que son entendidos por una persona. Pero en verdad, los hosts se identifican mediante direcciones IP.

Y acá entra la DNS.

DNS es una BDD distribuida implementada en una jerarquía de servidores DNS y un protocolo de capa de aplicación que permite a los hosts consultar la BDD distribuida.

- Se ejecuta sobre **UDP** y utiliza el puerto **53**.

Su función es traducir el nombre del host a la IP del host.

Servicios que proporciona

- **Alias de host:** un host con nombre complicado puede tener uno o más alias. Generalmente los nombres **canónicos** son los menos mnemónicos.
- **Alias al servidor de correo:** idem pero con los correos
- **Distribución de carga:** aplica un Round Robin sobre sus servidores para balancear la carga en las peticiones.

Base de datos jerárquica y distribuida

Existen 3 tipos de servidores:

- Raíz
- De dominio de nivel superior (TLD, Top-Level Domain)
- Autoritativos

El orden de la consulta es Raíz → TLD → Autoritativo.

También se cuenta con un **servidor DNS local** que actúa como primer contacto y mediador para cualquier consulta que realice una computadora. Inclusive el ISP cuenta con uno. Y funciona como un servidor **proxy**.

Tipos de consultas

Existen dos tipos de consultas:

- **Recursivas:** el host que realiza la petición delega toda la responsabilidad en el servidor DNS local. Se queda a la espera de que le llegue la respuesta.
- **Iterativas:** el servidor consultado no busca la respuesta final, sino que guía al solicitante en ella. Le responde con la dirección del siguiente servidor obligando a tener que realizar la siguiente llamada él mismo.

Tipos de registros DNS

Define lo que se puede pedir en una consulta DNS.

1. A. Consulta por una IPv4.

2. AAAA. Consulta por una IPv6.
3. CNAME. Consulta por alias de un dominio.
4. MX. Consulta por el servidor de correo del dominio.

Cache DNS

Cuando un servidor DNS recibe una respuesta DNS puede almacenar esta información en su cache.

Tiene ventajas como:

- Reducir la latencia
- Reducir el tráfico de red
- Autonomía y disponibilidad
- Eficiencia